

-) **3V-Volume**: la quantité massive de données générées chaque jour. -**variété**: la diversité des types de données (20% structures et 80% non structures). -**vélocité**: Vitesse, c'est la rapidité à laquelle les données sont générées, collectées et traitées.

-) **hdfs dfs -setrep 4 test.txt**

-) Cette commande "**hdfs dfs -copyFromLocal -f file.txt /dossier**" copie le fichier "file.txt" depuis le système de fichiers local vers le dossier spécifié « dossier » dans le système de fichiers Hadoop.

-) Cette commande "**hdfs dfs -copyToLocal /user/root/file.txt hdfs**" copie le fichier "file.txt" depuis le système de fichiers Hadoop (HDFS) vers le répertoire local spécifié « hdfs ».

-) **4 étapes dans un traitement MapReduce**: 1/Découper (split) les données d'entrées en plusieurs fragments. 2/ Mapper chacun de ces fragments pour obtenir des couples (clef/Valeur). 3/grouper(shuffle) ces couples(clef/Valeur) par clef. 4/ réduire (reduce) les groupes indexés par clef en une forme finale, avec une Valeur pour chacune des clefs distinctes.

Avantage MapReduce: Scalabilité – Flexibilité – Rapide – Cost effective – haute disponibilité.

-) **Apache Hive** : est un système qui permet d'analyser des ensembles de mégadonnées et de maintenir des métadonnées stockées dans hdfs, il utilise une BDR appelée metastore pour assurer la persistance de ces données aussi il utilise un langage similaire à sql appelé HiveQL pour manipuler ces données.

-) **FSImage**: est un fichier stocké dans le système de fichiers local du namenode, il a pour fonction de stocker la structure organisationnelle de tous les fichiers et répertoires, aussi leurs propriétés telles que l'espace disponible, la taille...

Edit Log: est un fichier journal de transactions utilisé pour un enregistrement persistant à chaque changement que subissent les métadonnées du système de fichiers.

Dans le cas d'une panne de courant sur le namenode il faut effectuer une récupération en utilisant FSImage et editLog.

-) **Managed Table** : la suppression d'une managed table entraîne la suppression des métadonnées ainsi que les données dans HDFS

-) **External Table** : la suppression d'une external table entraîne uniquement la suppression des métadonnées.

-) **Partition une table dans hive** : la partition d'une table hive consiste à diviser une table en plusieurs parties logiques appelées partitions qui peuvent être stockées séparément et gérées indépendamment.

-) **Hbase** : est un système de gestion de BD non relationnelles distribué disposant d'un stockage structuré pour les grandes tables, stocké sur HDFS sous forme de fichier appelé Hfile.

-) **Mécanisme de stockage dans hbase** : 1/Row key : chaque ligne du tableau hbase est identifiée de façon unique par une clé « row key ». 2/famille de colonnes : chaque famille de colonnes est persistée dans Hfile séparé. 3/ la colonne : c'est l'adresse d'une série de données dans une famille de colonnes. 4/ cellule ou valeur : est l'intersection entre une row key, une famille de colonnes et une colonne, les données sont stockées dans les cellules. 5/ Time stamp : c'est la date et l'heure, à la milliseconde près, auxquelles la mise à jour a été effectuée.

-) **var** est mutable, ce qui signifie que vous pouvez réassigner une nouvelle valeur à cette variable après son initialisation.**val** est immuable, ce qui signifie qu'une fois qu'elle est initialisée avec une valeur, cette valeur ne peut pas être modifiée.

-) **Non**, Scala ne prend pas en charge les instructions "break" et "continue" dans les boucles "for" de la même manière que certains autres langages de programmation le font, comme Java. Cependant, vous pouvez utiliser des alternatives pour obtenir des fonctionnalités similaires.

-) **Une classe singleton** est une classe qui ne peut être instanciée qu'une seule fois. Cela signifie qu'il n'existe qu'une seule instance de cette classe dans tout le programme et l'utilisation de classes singleton est courante en Scala pour implémenter des objets uniques et globaux dans une application.

-) **un objet singleton** est un objet unique qui est défini en utilisant le mot-clé Object ne peut pas être instancié avec le mot clé new, utilisation pour regrouper des méthodes et des propriétés liées de manière cohérente.

-) **un objet compagnon** est un objet qui est défini dans le même fichier source qu'une classe et partage le même nom que cette classe et sont utilisés pour stocker des méthodes et des champs statiques. Les objets compagnons ne peuvent pas avoir des constructeurs, mais ils sont souvent utilisés avec une méthode apply.

-) **une classe compagnon** est une classe qui est associée à un objet compagnon. Les classes compagnons et les objets compagnons sont déclarés dans le même fichier source et ont le même nom. Cette relation particulière entre une classe et son objet compagnon permet un accès facile aux membres privés de la classe, ainsi que la définition de méthodes et de membres statiques partagés.

-) **une fonction anonyme** est une fonction qui n'a pas de nom, peuvent être utilisées de manière similaire aux fonctions. (parameters)=> expression
-) **Case classe** Contient juste des variables ou des champ conçues pour la manipulation des données immuables avec Les méthodes automatique comme apply, copy, equals....
-) **les traits** sont un concept fondamental de la programmation orientée objet qui fournit un mécanisme pour réutiliser le code. Sont utilisés comme des modules pour les classes en scala.
-) **RDD** : c'est l'ensemble des données distribuées dans les nœuds du cluster
-) **Scala** : est un langage de programmation fonctionnelle similaire au java et ces caractéristiques sont : POO et évolutive
-) **Resilient Distributed Dataset (RDD)** :
 1. **Resilient** : supporte la tolérance aux pannes et au recalcul les partitions manquantes.
 2. **Distributed** : données distribuées sur les nœuds du cluster.
 3. **Dataset** : collection de données partitionnée.
-) **port accès web interface 4040**
-) **Transformations**: Manipule un RDD, retourne un autre RDD Exemples : map() mapToPair() filter() groupByKey() reduceByKey() join(), cogroup()... **Actions** Ne retournent pas un RDD mais retourner à une Valeur Exemples : count() saveAsHadoopFile() foreach() collect().
-) **DataFrame** est une structure de données tabulaire distribuée dans Spark, offrant un niveau d'abstraction plus élevé que les RDD (Resilient Distributed Dataset) pour manipuler des données structurées.
-) **Les transformations paresseuses (ou "lazy transformations")** sont un concept clé dans Apache Spark pour optimiser les performances et minimiser le coût de traitement des données.
-) **une vue temporaire (temporary view)** fait référence à une structure de données tabulaire temporaire qui est créée à partir d'un DataFrame ou d'une table et qui peut être utilisée pour effectuer des opérations de requête SQL ou des transformations DataFrame. Les vues temporaires sont souvent utilisées dans le contexte de Spark SQL pour manipuler des données à l'aide de requêtes SQL.
-) **Les User-Defined Functions (UDFs)** fournissent une extension très puissante à Spark SQL en permettant aux utilisateurs de définir leurs propres fonctions personnalisées pour traiter les données.
-) **Apache Spark** est un outil open source conçu pour l'exécution massive offrant un environnement efficace pour le traitement de données. Spark est souvent décrit comme un moteur d'analyse conçu pour le Big Data et l'apprentissage automatique.
-) **L'écosystème Apache Spark** est bâti sur un moteur d'exécution fondamental qui offre beaucoup de caractéristique . on a la disposition des bibliothèques importantes commeaussi le support des langages populaires tel quecomme il permet le stockage dans divers systèmes comme, HDFS, Amazon Cloud S3, et pour info Spark prend aussi en charge les bases de données SQL et NoSQL.
-) **les _ (soulignés)** peuvent être utilisés comme des espaces réservés dans les paramètres de fonction des fonctions anonymes. Ils sont largement utilisés dans Spark, surtout lorsque les types de paramètres sont déjà connus.
-) **Les tableaux** en Scala représentent plusieurs valeurs du même type. Ils sont déclarés avec le mot-clé `Array` et le type est enclos entre crochets `[]`. Les éléments du tableau sont référencés avec des parenthèses `()`.
-) **Les collections** en Scala sont des conteneurs d'objets. Elles comprennent des types tels que les listes, ensembles, maps, tuples, options et itérateurs, qui sont tous pris en charge pour stocker et manipuler des données de manière efficace.
-) **Les listes** en Scala sont similaires aux tableaux mais sont représentées sous forme de listes chaînées, avec un nombre variable d'éléments. Elles prennent en charge des opérations telles que `head`, `tail`, `isEmpty`, `concat`, `contains`, qui permettent de manipuler les éléments de la liste de manière efficace.
-) **Les `Map`** en Scala sont des collections de paires clé-valeur. Elles permettent d'utiliser l'opérateur `+` pour ajouter de nouveaux éléments à la collection. Les `Map` prennent en charge des méthodes telles que `keys`, `values`, `isEmpty`, `contains`, `get` pour manipuler les données stockées dans la map.
-) **Programme pilote (Driver Program)** : est le processus de contrôle principal dans les applications Spark. Il exécute la fonction principale de l'application et crée le SparkContext et gère cycle de vie d'application.
-) **SparkContext** : point d'entrée de l'écosystème Spark. Il établit une connexion avec l'environnement d'exécution Spark.
-) **Exécuteur (Executor)** : sont des nœuds de travail dans le chargé d'exécuter les tâches. Chaque application Spark a son propre ensemble d'exécuteurs.

-) **Tâche (Task)** : Une tâche est une unité de travail envoyée à un exécuteur pour exécution.

-) **Bloc (Block)** : unité de données immuable qui est stockée en mémoire ou sur le disque.

-) **Cache** : Le cache est souvent utilisé pour stocker des RDD ou des résultats intermédiaires de calculs qui sont susceptibles d'être réutilisés plusieurs fois.

*) **Transformations**: Manipule un RDD, retourne un autre RDD:

map() : une valeur → une valeur /mapToPair() : une valeur → un tuple /filter() : filtre les valeurs/tuples /groupByKey() : regroupe la valeurs par clés/reduceByKey() : aggrège les valeurs par clés /join(), cogroup()... : jointure entre deux

*) **Actions (RDD)**

finales : Ne retournent pas un RDD Exemples :

count() : compte les valeurs/tuples

saveAsHadoopFile() : sauve les résultats au format Hadoop

foreach() : exécute une fonction sur chaque valeur/tuple

collect() : récupère les valeurs dans une liste (List< T >)

-) **Interfaces SQL** qui permettent aux utilisateurs d'exécuter des requêtes SQL sur leurs données Spark SQL :

-) **Spark SQL** est un module intégré à Apache Spark qui permet d'exécuter des requêtes SQL sur des données structurées, ainsi que de manipuler des DataFrame et des DataSet à l'aide de la syntaxe SQL. Spark SQL prend en charge divers formats de données, JSON, Parquet, CSV

-) **Hive QL** : Spark peut être intégré à Apache Hive, En intégrant Spark avec Hive, les utilisateurs peuvent exécuter des requêtes HiveQL sur leurs données Spark et bénéficier de l'optimisation de requêtes avancée fournie par Hive.

-) **Les DataFrames Parquet** sont des DataFrames dans Spark dont les données sont stockées au format Parquet.

-) **Format Parquet** : est un format de stockage de données colonnaires, conçu pour être efficace en termes de stockage et d'accès aux données pour DataFrames Spark.

-) **Les opérations DataFrames**

*) **Transformations** :

Les transformations sont des opérations paresseuses. Elles ne sont pas immédiatement exécutées, mais forment plutôt un plan d'exécution.

Elles produisent un nouveau DataFrame. Exemples de transformations : select(): Sélectionne des colonnes.

filter(): Filtre les lignes selon une condition.

groupBy(): Regroupe les données en fonction de certaines clés.

join(): Joins deux DataFrames.

withColumn(): Ajoute une nouvelle colonne.

drop(): Supprime des colonnes.

orderBy(): Trie les données selon une ou plusieurs colonnes.

*) **Actions** :

Les actions déclenchent l'exécution des transformations et produisent un résultat concret (par exemple, une sortie écrite sur le disque ou une valeur retournée).

Elles déclenchent l'exécution du plan d'exécution construit par les transformations. Exemple d'actions :

show(): Affiche les premières lignes du DataFrame.

collect(): Récupère toutes les lignes du DataFrame en tant que tableau dans le programme driver.

write(): Écrit le DataFrame dans un support de stockage externe (par exemple, HDFS, S3, JDBC).

count(): Compte le nombre de lignes dans le DataFrame.

saveAsTable(): Sauvegarde le DataFrame sous forme de table dans une source de données externe (par exemple, Hive, HBase).

-) **la fonction d'agrégation agg()**, nous pouvons calculer plusieurs agrégations à la fois sur une seule instruction en utilisant les fonctions d'agrégation Spark SQL sum(), avg(), min(), max() moyenne()...

-) **les opérations de fenêtrage** font généralement référence à des opérations effectuées sur une fenêtre glissante ou une fenêtre fixe de données. Ces opérations sont couramment utilisées dans le traitement et l'analyse de données en continu ou de séries temporelles.

-) **dataframe parallelism** : Le parallélisme dans le contexte des dataframes fait généralement référence à la capacité d'effectuer des opérations sur les données simultanément sur plusieurs processeurs ou cœurs.

-) temporary view

*) **Une vue temporaire ordinaire** est créée à partir d'un DataFrame pour être utilisée dans une session Spark spécifique. Cette vue existe uniquement pendant la durée de cette session et disparaît lorsque la session prend fin.

*) **Une vue temporaire globale**, en revanche, persiste au-delà de la session qui l'a créée. Elle ne disparaît que lorsque l'application Spark entière prend fin

-) **Dataframe Caching**: Le caching des données dans Apache Spark est un mécanisme qui permet de stocker le contenu d'un DataFrame ou d'une table en mémoire, ou éventuellement sur le disque, afin d'accélérer les opérations ultérieures qui utilisent ces données.

-) **Spark web interface** : port (4040)

-) **Les Datasets** dans Apache Spark représentent une forme généralisée des DataFrames, permettant de manipuler des données de manière typée.

-) SQL Queries:

Select/Insert/Delete/Create Table

-) **Les User-Defined Functions (UDFs)** fournissent une extension très puissante à Spark SQL en permettant aux utilisateurs de définir leurs propres fonctions personnalisées pour traiter les données.

-) **Spark peut utiliser YARN** comme gestionnaire de ressources pour la planification et l'exécution des jobs

-) **master yarn** est utilisé pour déployer des applications Spark sur un cluster YARN, tandis que --master local est utilisé pour exécuter des applications localement sur la machine où spark-submit est lancé.

-) **deploy-mode** client et --deploy-mode cluster déterminent où le processus principal de l'application Spark s'exécute : localement sur la machine cliente ou sur un nœud du cluster Spark, respectivement.

```
//Example 1
object HelloWorld {
  def main(args: Array[String]) {
    println("Hello, world!")
  }
}

HelloWorld.main(null)
```

```
//Example 2 Scala components
class IntStack {
  var Ilist : List[Int] = List();
  def push(x:Int) ={
    Ilist = x :: Ilist
  }
  def pop() : Int = {
    var x = Ilist.head
    Ilist = Ilist.tail
    return x
  }
}
var Stk = new IntStack
Stk.push(20)
Stk.pop()
```

```
//Example 3      Datatypes
var I : Int = 100
var d : Double = 1.04
var s : String = "abc"
var c : char = 'A'
```

```
//Example 4  Variables
var I : Int = 100
I = 200
var k = 100L
var j = k + 100
val d : Double = 1.04
val s : String = "abc"
s = "xyz"
var t = (k,s)
```

//Example 5 Operators

```
var a = 30
var b = 7
var c = a % b
c += b
var d = (a > b) && (b > c)
```

```
//Example 6  Statements
def incr(a : Int) : Int = {
  var b = 2 * a
  b += 1
  return b
}
```

//Example 7 Conditional Statements

```
var max = 0
val a = 1
val b = 5
if (a > b) {
  max = a
} else {
  max = b
}
```

```
//Example 8 Loop statements
var i = 0
var a : Array[Int] = (0 to 20).toArray
a(3) = 100
do {
    print(a(i) + ",")
    i += 1
} while (i < 20)
println(i)
```

```
// Example 9 For loop
for(i <- 0 to 10) println(i)
for(i <- 0 until 10) println(i)
```

```
// Example 10 Functions
def max(a : Int, b: Int) : Int = {
    if(a>b) {
        a
    } else {
        b
    }
}
var c = max(10,20)
```

```
def max(a : Int, b: Int) : Int = {
    if(a>b) {
        a
        println("max = " + a)
    } else {
        b
        println("max = " + b)
    }
}
```

```
//Example 16 List
var mylist = List(1,2,3,4)
println(mylist.head)
println(mylist.tail)
println(mylist.isEmpty)
mylist = 0 :: mylist
mylist = mylist :+ 5
println(mylist)
```

```
//Example 17 Map
var Ages : Map[String,Int] = Map()
println(Ages.isEmpty)
Ages += ("John" -> 20)
Ages += ("Chris" -> 30)
println(Ages.keys)
println(Ages.values)
if(Ages.contains("John"))
    println(Ages("John"))
```

```
// Example 11 Anonymous functions
var add = (x :Int, y: Int) => x + y
var z = add(14,15)
var x = add(3,5)
var y = add(5,8)
```

```
//Example 12 Closure
var z = 10
var addz = (x :Int) => x + z
var x = addz(14)
var z = 20
var x = addz(3)
```

```
//Example 13 Arrays
var tab : Array[Int] = new Array[Int](10)
for(i<- 0 until tab.length)
    tab(i) = i * 2
for( i <- tab) println(i)
```

```
//Example 14 Strings
var A = "Hello"
var B = " World"
var C = A + B
printf(" C = %s\n", C)
```

```
//Example 15 Collections
var week = List("Sun","Mon","Tue","Wed","Thu","Fri","Sat")
var monthEnd = Set(28,29,30,31)
var age = Map("John"->30, "Linda"->25)
var tup = ("John", 30)
var withinTen : Option[Int] = Some(10)
```

```
//Example 18 Classes and objects
class IntStack {
protected var Ilist : List[Int] = List();
  def push(x:Int) ={
    Ilist = x :: Ilist
  }
  def pop() : Int = {
    var x = Ilist.head
    Ilist = Ilist.tail
    return x
  }
}
var Stk = new IntStack
Stk.push(20)
Stk.push(30)
Stk.push(40)
println(Stk.pop())
println(Stk.pop())
```

```
//Example 19 Class Extension
class newStack extends IntStack{
  override def toString():String = {
    "IntStack[" + Ilist.toString + "]"
  }
}
var Stk = new newStack
Stk.push(20)
Stk.push(30)
Stk.push(40)
println(Stk)
println(Stk.pop())
println(Stk)
```

```
// Example 22 traits
trait mapTrait[T] {
  var tmap : Map[String,T] = Map()
  def add(x:String, y:T) = {
    tmap = tmap.+(x->y) }
  def get(x:String) : T = {
    return tmap(x)
  }
}
class mapStr extends mapTrait[String]
class mapInt extends mapTrait[Int]
```

```
var mstr = new mapStr()
mstr.add("a","abc")
mstr.add("b","pqr")
mstr.get("a")

var mint = new mapInt()
mint.add("a",100)
mint.add("b",200)
mint.get("a")
```

```
//Example 20 Companion class
class thisYear {
  var year: Int = 0
  def get(): Int = { year }
}
object thisYear {
  def apply(x:Int) : thisYear = {
    val y = new thisYear()
    y.year = x
    return y
  }
}
val x = thisYear(2020)
x.get
val y = thisYear(2019)
y.get
```

```
//Example 21 Case class
case class stock (ticker:String, price: Int)
val a = stock("ABC",100)
val b = a.copy(price=200)
val c = a == b
println(b)
a match {
  case stock("ABC", 200) => println("ABC",200)
  case stock("ABC",x) => println("x="+ x)
  case _ => println("None of the above")
}
```

```
//Example 23 functions as parameters
def filter( f: Int=>Boolean, i : Int ): Boolean = {
  f(i)
}
filter((x:Int)=>x>20, 30)
filter((x)=>x>20,30)
filter(x=>x>20,30)
filter(_>20,30)
filter(_>20,10)
```

Ces appels utilisent différentes formes pour définir la fonction qui est passée en tant que premier argument à la fonction `filter`.

- `filter((x: Int) => x > 20, 30)`: Utilise une fonction anonyme avec une signature explicite.
- `filter((x) => x > 20, 30)`: Omet la signature du paramètre, Scala peut l'inférer.
- `filter(x => x > 20, 30)`: Si la fonction a un seul paramètre, vous pouvez omettre les parenthèses autour du paramètre.
- `filter(_ > 20, 30)`: Utilise la notation de paramètre de sous-ligne (`_`) pour rendre le code plus concis.

Chaque appel teste si l'entier passé en deuxième argument satisfait la condition spécifiée par la fonction.

- Le premier, deuxième et troisième appel renverront `true` car `30 > 20`.
- Le quatrième appel utilise la notation de sous-ligne, et il renverra également `true`.
- Le cinquième appel renverra `false` car `10` n'est pas supérieur à `20`.